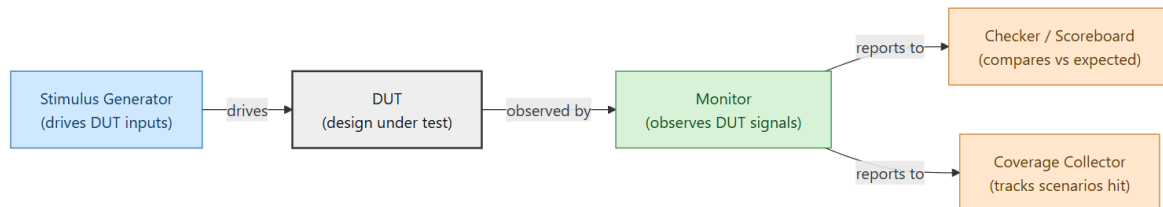


Testbench Anatomy

Block Diagram

Day 3 Experimental Work — Item 2: “Draw the anatomy of a testbench — a block diagram with DUT in the centre and all testbench components around it, with arrows showing data/signal flow.”

Figure 1 — Testbench anatomy: DUT wrapped by the testbench components



Legend: grey = DUT (neutral, unmodified) · blue = drives the DUT · green = observes the DUT · orange = analyzes what the monitor saw.

Data / signal flow, explained

1. **Stimulus generator → DUT.** The stimulus generator drives the DUT’s input ports — this is the only component that ever *writes* to the DUT.
2. **DUT → Monitor.** The monitor passively watches the DUT’s input and output ports. It never drives anything; it only observes and forwards what it sees.
3. **Monitor → Checker / Scoreboard.** The monitor’s observations are forwarded to the checker, which compares what the DUT actually produced against what a reference/golden model says it *should* have produced, and flags any mismatch.
4. **Monitor → Coverage Collector.** The same observations are also forwarded to the coverage collector, which tracks which scenarios (states, values, combinations of signals) have actually been exercised so far in the run.

Why the DUT sits in the middle

The DUT is drawn as a plain, neutral block deliberately: none of the surrounding testbench logic (stimulus, monitor, checker, coverage) ever lives *inside* the DUT or changes it. All the “intelligence” of verification — driving the right inputs, watching the right outputs, deciding pass/fail, tracking what’s been tested — lives in the testbench code that *wraps around* the DUT, never in the DUT itself.

This is what Learning Topic 3.3 means by “the testbench wraps the DUT,” and it’s also why a DUT can be swapped in/out of different testbenches (or a testbench reused across DUT revisions) without either side needing to know the internal details of the other.